

Python Topic 20.1 — Processes vs Threads & CPU Limits

Full Stack Python + AI Roadmap • Companion to Topic 20 (Concurrency) • Definitions + how many threads/processes you can actually use



Part A — The Building Blocks: Process vs Thread

All of concurrency is built from two OS-level concepts: the **process** and the **thread**.

Process

A **process** is a running program — an independent instance of your code that the OS loads into memory and executes. Running `python3 myscript.py` creates *one process*.

- Has its **own private memory space** — walled off from other processes
- Its own copy of the interpreter, resources, and file handles
- **Isolated** — one process crashing doesn't take down another
- **Heavy** — creating one costs time and memory

Think of a process as a **separate house**: its own rooms, its own furniture, walled off from neighbors.

Thread

A **thread** is a unit of execution *inside* a process. Every process has at least one (the "main thread"). A thread is the actual sequence of instructions being run.

- Lives **inside** a process and **shares that process's memory** with sibling threads
- **Lightweight** — cheap to create compared to a process
- Shares data easily with other threads — powerful *and* dangerous (race conditions)

Think of threads as **people working inside one house**: they share the same rooms and furniture (memory), so they hand things to each other instantly — but they can also collide.

Process (a house)

```
|— Thread 1 (person)  └─
|— Thread 2 (person)  └─ all share the same memory/rooms
|— Thread 3 (person)  └─
```

The four terms, defined

- **Threading** — the general technique of using threads (Python's `threading` module).
- **Multithreading** — running **multiple threads at once** inside one process. In practice "threading" and "multithreading" are used interchangeably; "multi" just emphasizes several threads run concurrently. One process, many threads, **shared memory**, GIL-limited for CPU work.
- **Process** (as a technique) — running work as a separate program instance with its own memory, versus a thread that shares memory.

- **Multiprocessing** — running **multiple processes at once** (Python's `multiprocessing` module), each with its own interpreter and memory, each with its own GIL → **true parallelism** across cores.

Side-by-side

	Multithreading	Multiprocessing
Unit	Multiple threads	Multiple processes
Memory	Shared (one space)	Separate (isolated)
Weight	Lightweight	Heavy
Data sharing	Easy — but needs locks	Hard — Queue/Pipe, pickled
GIL	Limited by it (1 thread runs Python at a time)	Bypasses it (own interpreter each)
True parallelism?	No (for Python bytecode)	Yes (multiple cores)
A crash...	can affect the whole process	is isolated to that process
Best for	I/O-bound (waiting)	CPU-bound (computing)

i The house analogy, complete: **Multithreading** = one house, several people sharing all rooms (memory) — fast to add a person, easy to share, but they bump into each other, and only one can use the "Python kitchen" at a time (the GIL). **Multiprocessing** = several separate houses, each with its own kitchen — expensive to build, passing things means mailing packages (pickling), but everyone can cook at once (true parallelism).

Part B — Are There Limits? How Many Can You Create?

Two different questions: how many you *can* create (hard OS/hardware limits), and how many are *worth* creating (practical limits). The practical limit almost always matters more.

1. The hardware limit — CPU cores

Your CPU has a fixed number of **cores**, often 2× that in **logical processors** (hyper-threading/SMT). This caps **true parallelism** — how many things genuinely run at the exact same instant.

```
import os
print(os.cpu_count())          # e.g. 8 → logical cores available
```

A 4-core / 8-thread CPU truly runs only **8** streams of computation at once. You can *create* far more threads/processes than cores, but the OS **time-slices** them — rapidly switching to give the *illusion* of simultaneity. So for CPU-bound multiprocessing, the sweet spot is about `os.cpu_count()` processes — one per core.

```
from concurrent.futures import ProcessPoolExecutor
import os
```

```
# ideal for CPU-bound work: match the core count
with ProcessPoolExecutor(max_workers=os.cpu_count()) as executor:
    ...
```

2. Practical limit for threads (I/O-bound work)

For threads doing I/O, core count matters far less — they spend most time *waiting*, not computing. You can profitably run **hundreds** of threads, since at any moment most are idle (waiting on network/disk) and have released the GIL. Ceilings that still apply:

- **Memory per thread** — each reserves a stack (commonly ~8 MB of virtual address space on Linux; only a fraction actually used). 1,000 threads = significant memory reserved. This is the practical cap.
- **Context-switching overhead** — more threads means the OS spends more time *switching* instead of working. Past a point, more threads *slow you down*.
- **The GIL** — for CPU work, extra threads give zero speedup, so never exceed core count for computation.

✓ **Rule of thumb:** I/O-bound → many threads are fine (often 10s–100s, tune to workload); CPU-bound → don't exceed core count.

3. Practical limit for processes

Processes are much heavier than threads:

- Each has its **own full memory space** + interpreter copy — memory is the main constraint. 100 processes × 200 MB = 20 GB RAM. You hit RAM limits fast.
- High startup cost (spawning a new interpreter).
- Passing data requires serialization (pickling) — more overhead.

So rarely want hundreds of processes. For CPU work, `os.cpu_count()` (or slightly less, leaving room for the OS) is the practical max that helps.

4. Hard OS limits (rarely hit, but real)

```
ulimit -u                # max user processes/threads
cat /proc/sys/kernel/threads-max  # system-wide thread max
ulimit -n                # max open file descriptors
```

The OS caps threads/processes per user and system-wide, plus file-descriptor limits. These are typically in the thousands to millions — you usually hit *practical* (performance/memory) limits long before these. Ultimately you run out of RAM before you run out of "permission."

Putting it together

Work type	Tool	How many?
CPU-bound (computing)	Multiprocessing	≈ <code>os.cpu_count()</code> — one per core; more is wasteful

I/O-bound (waiting)	Threading	10s–100s+ fine; tune to workload; watch memory
I/O-bound, huge scale	asyncio	Thousands of tasks on one thread — cheapest of all

i Why async wins for high-concurrency I/O: an async task is far lighter than a thread (no per-thread stack, no OS context-switch), so you can have tens of thousands of concurrent awaits on a single thread. For a FastAPI backend handling many simultaneous requests, that's the whole point.

Interview Q&A Cards

Q: Process vs thread — core difference?

A: A process is an independent program with its own private memory; a thread runs inside a process and shares that process's memory with sibling threads. Processes are heavy and isolated; threads are light and share state (so they need locks).

Q: Multithreading vs multiprocessing?

A: Multithreading = many threads in one process (shared memory, GIL-limited, good for I/O). Multiprocessing = many processes (separate memory, own GIL each, true parallelism, good for CPU).

Q: Is there a limit to how many threads/processes you can create?

A: True parallelism is capped by CPU cores, so CPU-bound processes should roughly match `os.cpu_count()`. Threads and async can far exceed core count because they mostly wait, but they're limited by memory and context-switching — threads in the hundreds, async tasks in the thousands. Hard OS limits exist but you usually hit memory first.

Q: How many processes for CPU-bound work, and why not more?

A: About one per core (`os.cpu_count()`). Beyond that, processes just contend for the same cores via time-slicing with no gain, while adding memory and context-switch overhead.

🔗 Memory hook: *Threads share one house's rooms (memory) → light, GIL-limited, good for waiting. Processes are separate houses (own memory + own GIL) → heavy, truly parallel, good for computing. Cores cap true parallelism → ~1 process per core; waiting work isn't core-limited → many threads (100s) or async tasks (1000s), capped by memory, not cores.*